

Introduction à Dabara

Bienvenue dans la documentation officielle de **Dabara**, le langage de programmation en haoussa !

Qu'est-ce que Dabara ?

Dabara (prononcé *dah-BAH-rah*) est un langage de programmation révolutionnaire qui utilise des mots-clés en haoussa, conçu pour démocratiser l'accès à la programmation pour les communautés Hausa d'Afrique de l'Ouest.

Philosophie

Dabara repose sur une philosophie simple mais puissante : **la technologie doit être accessible dans sa langue maternelle**. En utilisant des termes haoussa familiers, nous supprimons les barrières linguistiques qui empêchent souvent l'apprentissage de la programmation.

Objectifs

- **Accessibilité linguistique** : Permettre aux locuteurs haoussa d'apprendre la programmation dans leur langue
- **Préservation culturelle** : Valoriser et moderniser la langue haoussa dans le contexte technologique
- **Éducation** : Faciliter l'enseignement de la programmation dans les écoles d'Afrique de l'Ouest
- **Innovation** : Créer un écosystème de développement entièrement en haoussa

Caractéristiques principales

✨ Syntaxe naturelle

- Utilisation de mots haoussa pour tous les concepts de programmation
- Support complet d'Unicode pour les caractères spéciaux (ƙ, ɗ, ɓ, etc.)
- Syntaxe intuitive et logique

🚀 Performance

- Interpréteur optimisé écrit en Rust
- Gestion efficace de la mémoire

- Messages d'erreur clairs et localisés

Inclusivité

- Variants latins pour faciliter la saisie sur tous les claviers
- Documentation bilingue (français/haoussa)
- Communauté accueillante

Histoire du projet

Dabara est né de la conviction que la diversité linguistique enrichit le monde de la programmation. Le projet a démarré en 2024 avec l'objectif de créer le premier langage de programmation entièrement en haoussa.

Étymologie

Le nom "Dabara" vient du mot haoussa qui signifie "méthode" ou "façon de faire", reflétant parfaitement l'essence d'un langage de programmation.

Public cible

Débutants

- Étudiants haoussa souhaitant apprendre la programmation
- Enseignants cherchant des outils pédagogiques adaptés
- Développeurs juniors voulant commencer avec un langage familier

Développeurs expérimentés

- Développeurs souhaitant explorer de nouveaux paradigmes
- Contributeurs open source intéressés par l'innovation linguistique
- Chercheurs en informatique culturelle

Écosystème

Dabara fait partie d'un écosystème plus large comprenant :

- **Compilateur/Interpréteur** : Le cœur du langage

- **Documentation** : Guides complets et références
- **Outils de développement** : Éditeurs, debuggers, etc.
- **Communauté** : Forums, tutoriels, exemples

Prochaines étapes

Pour commencer votre voyage avec Dabara :

1. [Installer Dabara](#)
2. Suivre le [guide de démarrage rapide](#)
3. Explorer les [exemples](#)
4. Rejoindre la [communauté](#)

Sannu da zuwa cikin duniyar Dabara! (Bienvenue dans le monde de Dabara !)

“Programming is not about what you know; it’s about what you can figure out.” - Chris Pine

Avec Dabara, cette découverte se fait dans votre langue maternelle. 🌟

Installation de Dabara

Ce guide vous accompagne dans l'installation de Dabara sur votre système.

Prérequis système

Systèmes d'exploitation supportés

- **Linux** (Ubuntu 18.04+, Debian 10+, Fedora 30+, Arch Linux)
- **Windows** (Windows 10/11)
- **macOS** (10.15 Catalina ou plus récent)

Configuration minimale

- **RAM** : 512 MB
- **Espace disque** : 50 MB
- **Processeur** : x86_64 ou ARM64

Option 1 : Installation via binaires pré-compilés (Recommandé)

Étape 1 : Téléchargement

1. Rendez-vous sur la [page des releases](#)
2. Téléchargez l'archive correspondant à votre système :

Système	Architecture	Fichier
Linux (glibc)	x86_64	dabara-x86_64-unknown-linux-gnu.tar.gz
Linux (musl)	x86_64	dabara-x86_64-unknown-linux-musl.tar.gz
Windows	x86_64	dabara-x86_64-pc-windows-msvc.zip
macOS (Intel)	x86_64	dabara-x86_64-apple-darwin.tar.gz
macOS (Apple Silicon)	ARM64	dabara-aarch64-apple-darwin.tar.gz

Étape 2 : Extraction

Sur Linux/macOS

```
# Naviguez vers le dossier de téléchargement
cd ~/Downloads

# Extrayez l'archive
tar -xzf dabara-*.tar.gz

# Le contenu sera extrait dans un dossier dabara-*
```

Sur Windows

```
# Utiliser PowerShell ou l'explorateur Windows
Expand-Archive -Path "dabara-*.zip" -DestinationPath "."
```

Étape 3 : Installation dans le PATH

Sur Linux/macOS

```
# Option 1 : Installation système (nécessite sudo)
sudo mv dabara-*/dabara /usr/local/bin/

# Option 2 : Installation utilisateur
mkdir -p ~/.local/bin
mv dabara-*/dabara ~/.local/bin/

# Assurez-vous que ~/.local/bin est dans votre PATH
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Sur Windows

1. Copiez `dabara.exe` dans un dossier de votre choix (ex: `C:\\dabara\\`)
2. Ajoutez ce dossier à votre variable d'environnement `PATH` :
 - Ouvrez "Paramètres système avancés"
 - Cliquez sur "Variables d'environnement"
 - Modifiez la variable `PATH` et ajoutez le chemin vers dabara

Étape 4 : Vérification

Ouvrez un nouveau terminal et testez :

```
dabara --version
```

Vous devriez voir :

```
dabara 0.0.1
```

Option 2 : Compilation depuis les sources

Prérequis

- [Rust](#) 1.70 ou plus récent
- Git

Installation de Rust

```
# Installation de Rust via rustup
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
```

Compilation

```
# Cloner le repository
git clone https://github.com/votre-username/dabara.git
cd dabara

# Compiler en mode release
cargo build --release

# Installer dans ~/.cargo/bin
cargo install --path .
```

Vérification

```
dabara --version
```

Option 3 : Installation via Cargo

```
# Installation directe depuis crates.io (quand disponible)
cargo install dabara

# Ou depuis GitHub
cargo install --git https://github.com/votre-username/dabara.git
```

Configuration post-installation

Éditeur de texte

Pour une meilleure expérience de développement, configurez votre éditeur :

VS Code

1. Installez l'extension "Dabara Language Support" (quand disponible)
2. Ou configurez la coloration syntaxique pour les fichiers `.ha`

Vim/Neovim

```
" Ajoutez dans votre .vimrc
autocmd BufNewFile,BufRead *.ha set filetype=hausa
```

Emacs

```
;; Ajoutez dans votre configuration
(add-to-list 'auto-mode-alist '("\\.ha\\'" . text-mode))
```

Variables d'environnement

```
# Pour activer le mode debug
export DABARA_DEBUG=1

# Pour personnaliser les messages d'erreur
export DABARA_LANG=ha # ha pour haoussa, fr pour français
```

Vérification de l'installation

Créez un fichier test `sannu.ha` :

```
fara
  rubuta "Sannu duniya!"
kare
```

Exécutez-le :

```
dabara sannu.ha
```

Sortie attendue :

Sannu duniya!

Résolution des problèmes

Erreur : “command not found”

- Vérifiez que dabara est dans votre PATH
- Redémarrez votre terminal
- Sur Linux/macOS : `which dabara`
- Sur Windows : `where dabara`

Erreur de permissions

- Sur Linux/macOS : utilisez `chmod +x dabara`
- Sur Windows : exécutez en tant qu'administrateur

Problèmes de dépendances

- Sur Linux : installez `libc6-dev` ou `musl-dev`
- Utilisez la version musl si vous avez des problèmes avec glibc

Support

- [Issues GitHub](#)
- [Discussions](#)
- [Documentation complète](#)

Prêt pour la prochaine étape ? → [Démarrage rapide](#)

Démarrage rapide avec Dabara

Ce guide vous permettra d'écrire votre premier programme Dabara en quelques minutes !

Votre premier programme

Créer un fichier

Créez un nouveau fichier avec l'extension `.ha` :

```
touch sannu.ha
```

Écrire le code

Ouvrez le fichier et ajoutez :

```
fara
  rubuta "Sannu duniya!"
kare
```

Exécuter

```
dabara sannu.ha
```

Résultat :

```
Sannu duniya!
```

 **Félicitations !** Vous venez d'exécuter votre premier programme Dabara !

Structure d'un programme

Tout programme Dabara suit cette structure :

```
fara
  // Votre code ici
kare
```

- `fara` : Marque le début du programme

- `kare` : Marque la fin du programme
- Tout le code doit être entre ces deux mots-clés

Exemples de base

1. Affichage de texte

```
fara
  rubuta "Barka da zuwa!"
  rubuta "Ina kwana?"
kare
```

2. Variables

```
fara
  nada sunan = "Khadija"
  nada shekarun = 25
  rubuta "Sunanta: " + sunan
  rubuta shekarun
kare
```

3. Calculs simples

```
fara
  nada a = 10
  nada b = 5
  nada jimla = a kara b
  rubuta "Jimla: "
  rubuta jimla
kare
```

Concepts de base

Types de données

- **Nombres** : 42 , 0 , 999
- **Texte** : "Sannu" , "Barka da zuwa"
- **Booléens** : gaskiya (vrai), karya (faux)

Mots-clés essentiels

- `fara` : début de programme
- `kare` : fin de programme
- `rubuta` : afficher/imprimer
- `nada` : créer une variable
- `kara` : addition
- `rage` : soustraction

Support clavier

Si vous ne pouvez pas taper les caractères spéciaux, utilisez les variants latins :

- `kare` → `kare`
- `nada` → `nada`
- `kara` → `kara`

Exemples pratiques

Calculatrice simple

```
fara
nada prix1 = 1500
nada prix2 = 2000
nada total = prix1 kara prix2

rubuta "Prix 1: "
rubuta prix1
rubuta "Prix 2: "
rubuta prix2
rubuta "Total: "
rubuta total
kare
```

Présentation personnelle

```
fara
nada sunan = "Ahmad"
nada shekarun = 30
nada garin = "Kano"

rubuta "Sannu! Ni " + sunan
rubuta "Ina da shekaru "
rubuta shekarun
rubuta "Daga " + garin + " nake"
kare
```

Programme avec booléens

```
fara
nada yana_da_mota = gaskiya
nada yana_da_gida = karya

rubuta "Yana da mota: "
rubuta yana_da_mota
rubuta "Yana da gida: "
rubuta yana_da_gida
kare
```

Mode debug

Pour voir ce qui se passe dans votre programme :

```
export DABARA_DEBUG=1
dabara votre_programme.ha
```

Cela affichera :

- Le code source
- Les tokens générés
- L'arbre syntaxique (AST)

Gestion des erreurs

Dabara fournit des messages d'erreur clairs en haoussa :

Erreur de syntaxe

```
fara
rubuta "Oubli des guillemets
kare
```

Erreur :

```
Kuskure: Ana tsammanin alamar ''' a layi 2
```

Variable non définie

```
fara
  rubuta sunan_bai_wanzu
kare
```

Erreur :

Kuskure: Babu irin wannan mai canjin 'sunan_bai_wanzu'

Bonnes pratiques

1. Nommage des variables

```
//  Bon
nada sunan_makaranta = "Université de Kano"
nada yawan_dalibai = 5000
```

```
//  Éviter
nada x = "Université de Kano"
nada n = 5000
```

2. Organisation du code

```
fara
  // Déclaration des variables
  nada sunan = "Ahmad"
  nada shekarun = 25

  // Calculs
  nada shekarun_bana = shekarun kara 1

  // Affichage des résultats
  rubuta "Sannu " + sunan
  rubuta "Shekarunka bana: "
  rubuta shekarun_bana
kare
```

3. Commentaires (bientôt disponible)

```
fara
  // Wannan shi ne sunan mutum
  nada sunan = "Fatima"
  rubuta sunan
kare
```

Prochaines étapes

Maintenant que vous maîtrisez les bases :

1. **Explorez la [référence du langage](#)** pour découvrir toutes les fonctionnalités
2. **Consultez plus d'[exemples](#)** pour des cas d'usage avancés
3. **Rejoignez la [communauté](#)** pour partager vos créations

Aide et support

- **Documentation complète** : Parcourez toute la documentation
- **Exemples** : Section dédiée aux cas pratiques
- **FAQ** : Questions fréquemment posées
- **GitHub** : Signaler des bugs ou demander de l'aide

Bonne programmation avec Dabara ! 🚀

Premier programme écrit ? Félicitations ! Vous faites maintenant partie de la communauté Dabara. Continuez à explorer et n'hésitez pas à partager vos créations !

Vue d'ensemble

Lexer

Parser

Interpréteur

Gestion d'erreurs

Syntaxe de base

Types de données

Variables

Opérateurs

Mots-clés

Expressions

Commentaires

Hello World

Variables et calculs

Programmes complexes

Contribution

Architecture du code

Tests

Debugging

CLI

API interne

Extensions

Tableau de correspondance

Messages d'erreur

Glossaire

FAQ